Sparsity, Locality, and Persistent State in Modern Memory Architectures

As AI models move from processing short contexts to functioning as long-horizon agents, the way they handle memory must evolve. The dominant paradigm--the Transformer's Key-Value (KV) cache--stores an exact record of the past, but physically scales with sequence length. An analytical KV-cache estimate (assuming 12 layers, 8 heads, 64 dimensions per head, FP16 precision, and batch size 1, excluding model weights, activations, and allocator overhead) suggests that maintaining 100,000 tokens requires approximately 2.29 GiB of memory. This creates a bottleneck for exceptionally long contexts.

The alternative is maintaining a persistent state--a fixed-size memory that updates as new information arrives. But this introduces a profound challenge: writing new data into a fixed, globally connected state matrix can lead to overlapping associations and retrieval interference.

The Core Claim

Sparsity and locality can make persistent-state architectures more scalable by limiting how much of the state each update directly affects, but they introduce their own training and hardware-efficiency trade-offs.

The Mechanism of Interference

Imagine taking notes for an entire degree on a single sheet of paper by writing everywhere at once; the ink will blend into an unreadable mess. In neural architectures, this happens mathematically: if each new association writes broadly into a dense state matrix, previously stored vector representations can be continuously overwritten and corrupted by interference.

What Didn't Work: The Dense Recurrent Failure

Our toy experiment shows that this particular dense outer-product associative memory can suffer severe retrieval interference as associations accumulate or keys become correlated. In our simulation of an 8×8 dense state, storing 20 facts with 0% shared-key correlation achieves a mean recall of 38.0%, dropping to 9.0% at 80% key correlation (our experiment; seeds 0-9). (Note: because 20 keys cannot be made pairwise orthogonal in only 8 dimensions, some baseline interference exists even at 0% shared component--the correlation slider adds interference on top of this geometric floor; it does not start from a perfectly clean baseline.) The toy experiment demonstrates retrieval interference in a specific mathematical model, illustrating the difficulty of densely packing associations into a bounded space.

Locality as a Solution

A structural solution to this interference is locality. In neural architectures, locality means a specific input activates and updates a targeted subset of the network. When the model learns a new fact, it only alters the local state of a few parameters.

Evidence from the Frontier

This shift toward selective and local states maps across recent frontier architectures.

For instance, the selective state space model Mamba makes key state-space parameters input-dependent, allowing the model to selectively propagate or forget information along the sequence [Gu & Dao, 2023].

A more structural application of locality is found in the Dragon Hatchling (BDH) architecture [Kosowski et al., 2025]. BDH uses a scale-free network of locally interacting neuron particles with sparse positive activations [Kosowski et al., 2025]. BDH maintains evolving synaptic state within a recurrent network rather than allocating a new memory slot for every token. This architecture localizes state changes to particular synaptic interactions, providing a concrete alternative to globally updating a dense associative matrix.

Building on this, the BDH-CQ system uses recurrent contextual memory, updating that memory with demonstrations and inference-time inputs, then uses the compressed state for latent iterative computation [Engdahl et al., 2026]. While BDH and BDH-CQ are advanced research systems, they highlight how targeted, sparse updates can manage information accumulation in recurrent designs.

Limitations and Open Questions

While sparsity and locality mitigate retrieval interference in theory, they introduce their own trade-offs. Modern AI accelerators (GPUs and TPUs) are explicitly optimized for large, dense matrix multiplications. They are notoriously inefficient at handling sparse, localized memory reads and writes. A significant open question remains: can we design hardware or low-level CUDA kernels that execute sparse graph updates as efficiently as dense attention?

My Own Judgment

Long-horizon AI can benefit from persistent memory, and sparsity is one promising way to make such memory more scalable. We cannot continually pack information into globally updated dense matrices without risking retrieval interference. BDH explores a computational architecture built around sparse, localized activity and synaptic plasticity, offering one research direction for persistent state. I believe that until the hardware-software mismatch regarding sparse processing is resolved, the field will remain stuck in a compromise between memory efficiency and hardware utilization.

References

1. Kosowski, A., Uznanski, P., Chorowski, J., Stamirowska, Z., & Bartoszkiewicz, M. (2025). The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507.
2. Engdahl, B., Kosowski, A., Chorowski, J., Stamirowska, Z., Uznanski, P., Jiang, J., Phadke, R., Kinas, R., & Zhong, R. (2026). BDH-CQ: In-Context Learning with Recurrent Latent Reasoning. arXiv:2608.09888.
3. Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.